

```
1 from itertools import combinations
2
3 numbers = [3, 34, 4, 12, 5, 2]
4 target = 9
5
6 found = False
7
8 for r in range(1, len(numbers) + 1):
9     for subset in combinations(numbers, r):
10         if sum(subset) == target:
11             print("Subset found:", list(subset))
12             print("Sum:", sum(subset))
13             found = True
14             break
15     if found:
16         break
17
18 if not found:
19     print("No subset found")
```

Subset found: [4, 5]
Sum: 9

=== Code Execution Successful ===

```

20         best_distance = total
21         best_path = path
22
23     return best_distance, best_path
24
25
26 cities1 = [(1, 2), (4, 5), (7, 1), (3, 6)]
27 cities2 = [(2, 4), (8, 1), (1, 7), (6, 3), (5, 9)]
28
29 d1, p1 = shortest_path(cities1)
30 d2, p2 = shortest_path(cities2)
31
32 print("Test Case 1:")
33 print("Shortest Distance:", d1)
34 print("Shortest Path:", p1)
35
36 print("\nTest Case 2:")
37 print("Shortest Distance:", d2)
38 print("Shortest Path:", p2)

```

```

* Test Case 1:
Shortest Distance: 16.969112047670894
Shortest Path: [(1, 2), (7, 1), (4, 5), (3, 6), (1, 2)]

Test Case 2:
Shortest Distance: 23.12995011084934
Shortest Path: [(2, 4), (6, 3), (8, 1), (5, 9), (1, 7), (2, 4)]

=== Code Execution Successful ===

```

```

24
25     return best_assignment, best_cost
26
27
28 cost_matrix = [
29     [3, 10, 7],
30     [8, 5, 12],
31     [4, 6, 9]
32 ]
33
34 assignment, cost = assignment_problem(cost_matrix)
35
36 print("Optimal Assignment:")
37
38 for worker in range(len(assignment)):
39     print("Worker", worker + 1, "-> Task", assignment[worker]
40           + 1)
41
42 print("Total Cost:", cost)

```

* Optimal Assignment:

Worker 1 -> Task 3

Worker 2 -> Task 2

Worker 3 -> Task 1

Total Cost: 16

=== Code Execution Successful ===

```

29
30     if is_feasible(items, weights, capacity):
31         value = total_value(items, values)
32
33     if value > best_value:
34         best_value = value
35         best_items = list(items)
36
37     return best_items, best_value
38
39
40 weights = [2, 3, 1]
41 values = [4, 5, 3]
42 capacity = 4
43
44 items, value = knapsack(weights, values, capacity)
45
46 print("Optimal Selection:", items)
47 print("Total Value:", value)

```

```

Optimal Selection: [1, 2]
Total Value: 8

```

```

=== Code Execution Successful ===

```

```
4 numbers = [15, 10, 12, 7, 5]
5 target = 22
6
7 found = False
8
9 for r in range(1, len(numbers) + 1):
10     for subset in combinations(numbers, r):
11
12         if sum(subset) == target:
13             print("Subset found:", list(subset))
14             print("Sum:", sum(subset))
15             found = True
16             break
17
18 if found:
19     break
20
21 if not found:
22     print("No subset found")
```

Subset found: [15, 7]

Sum: 22

=== Code Execution Successful ===